VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA", BELAGAVI 590018, KARNATAKA, INDIA

A Synopsis on

O.A.S.I.S: Open Source Audio Visual Summarization and Insight System

Submitted for the Major Project Work Phase 1

(Academic Year 2025–26)

By

Shravan Kumar Soni	1NT23IS202
Srusti S	1NT23IS219
Surendra Tripathi	1NT23IS226
Vivek Singh	1NT23IS251

Under the Guidance of

Mr. Prashanth BS

Assistant Professor

Department of Information Science & Engineering

Nitte Meenakshi Institute of Technology, Bengaluru 560064

NITTE EDUCATION TRUST | NITTE MEENAKSHI INSTITUTE OF TECHNOLOGY

Declaration by Students

We, the undersigned students Shravan Kumar Soni (1NT23IS202), Srusti S (1NT23IS219), Surendra Tripathi (1NT23IS226) and Vivek Singh (1NT23IS251) hereby declare that we are fully aware of the project conduction procedures and evaluation rubrics as prescribed by the department. We understand that the project work will be evaluated through continuous assessment across the defined phases.

Name of the Student	**Signature**
Shravan Kumar Soni	
Srusti S	
Surendra Tripathi	
Vivek Singh	

Confirmation by Guide

I, Mr. Prashanth BS, hereby declare that I shall guide the above mentioned students in the project entitled "O.A.S.I.S: Open Source Audio Visual Summarization and Insight System" and will ensure that the project work is carried out under my supervision and submitted as per the academic requirements for the academic year 2025–2026.

Signature of the Guide

ABSTRACT

Long form video recordings such as business meetings, lectures, and documentaries serve as valuable knowledge repositories, but extracting information from them remains highly inefficient because users must manually scrub through long timelines to track specific speakers, topics, or on screen events. Existing automated tools usually ignore visual context, lose chronological consistency over long durations, or generate generic summaries that fail to answer a user's specific question. To address these gaps, this work introduces O.A.S.I.S, the Open Source Audio Visual Summarization and Insight System. The system processes audio tracks and video frames together, maps spoken content to visual changes, and creates a time anchored searchable representation of the video. Its query driven architecture produces customized answers instead of fixed summaries, transforming dense long form recordings into interactive and navigable knowledge assets.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter introduces the O.A.S.I.S project, the problem it addresses, the overall system design, and the technologies used in the prototype. It also outlines the structure followed in the rest of the report.

1.1 Overview and Need for the System

Long form video content such as business meetings, lectures, and documentaries is difficult to review efficiently. Identifying who said something, when it was said, and what was visible on screen at that time usually requires long periods of manual searching. Existing systems often focus only on transcripts, fail to preserve long range chronology, or produce generic summaries that do not answer user specific questions. O.A.S.I.S is designed to convert such recordings into time anchored, queryable knowledge assets using open source multimodal processing.

1.2 Pipeline Overview

The O.A.S.I.S prototype follows a six stage pipeline consisting of preprocessing, transcription with speaker diarization, visual scene understanding, multimodal fusion, model benchmarking, and dashboard based visualization. The six stage pipeline and its data flow are shown in Figure 5.1. Each stage produces structured outputs that are consumed by the next stage, making the system modular and easy to inspect.

1.3 Technology Stack

The system uses FFmpeg for video preprocessing, WhisperX and wav2vec2 for transcription and alignment, Pyannote for speaker diarization, Moondream2 for visual understanding, and open weight language models such as Qwen, Llama, Mistral, and Gemma for reasoning and summarization. PyTorch, Transformers, and bitsandbytes are used for model execution and quantized inference. Streamlit is used to build the dashboard interface for viewing outputs and benchmark metrics.

1.4 System Output and Utility

The final system produces timestamped transcripts, speaker identified dialogue, visual scene descriptions, model wise summaries, and benchmark statistics in a form that is easy to inspect. This makes the prototype useful not only for summary generation but also for reviewing meetings, tracing speaker contributions, and comparing open source model behavior on long videos.

1.5 Scope

The scope of O.A.S.I.S covers end to end processing of long form videos through audio extraction, speaker aware transcription, keyframe based visual context generation, multimodal fusion, comparative model benchmarking, and dashboard level visualization.

1.6 Organization of the Report

The report is organized as follows:

- Chapter 2 presents the literature survey, gaps, and motivation.
- Chapter 3 states the problem statement and objectives.
- Chapter 4 presents the software requirement and specification.
- Chapter 5 explains the proposed methodology for building the prototype.
- Chapter 6 provides the GitHub repository details.
- Chapter 7 includes the annexures.

Chapter 2

Literature Survey: Gaps and Motivation

This chapter reviews major research directions related to long form video understanding, meeting summarization, query based summarization, and benchmarking of large language models.

2.1 Long Form Video Language Understanding

2.1.1 HourVideo

HourVideo presents one hour video language understanding with questions on summarization, perception, and reasoning [4]. Strong proprietary systems perform only moderately well, while open models lag behind. The benchmark does not focus on speaker attribution or query driven meeting style understanding.

2.1.2 LongVideoBench

LongVideoBench evaluates long context video language understanding using interleaved video and text inputs [5]. Performance improves with more frames, but remains far from human level. It is not specifically designed for meetings or lectures with structured speaker context.

2.1.3 Scaling Up Video Summarization Pretraining

This work uses large language models to generate supervision for video summarization from ASR transcripts [11]. Larger summary datasets improve summarization quality. The method remains mainly transcript based and does not deeply integrate visual context.

2.2 Meeting and Lecture Summarization

2.2.1 MeetingBank

MeetingBank provides a benchmark of real meeting data with transcripts, agendas, and written minutes [6]. It is useful for long meeting summarization research. The dataset still remains

text centered and does not form a full audio visual query system.

2.2.2 QMSum

QMSum introduces query based meeting summarization where models locate relevant spans and answer queries [9]. Its formulation is closely related to this project. However, it uses transcripts only and ignores visual scene information.

2.2.3 MISP Meeting

MISP Meeting combines speech, vision, and text cues for long meeting transcription and summarization [7]. It shows that visual signals can improve downstream performance. It still does not provide a complete open query interface for practical use.

2.2.4 Generating Minutes of Meeting Using LLM

This work presents a practical pipeline for transcription, diarization, and minute generation using language models [2]. It demonstrates feasibility of automating documentation. It does not explicitly connect spoken content with visual scene changes.

2.3 Query Focused Video Summarization

2.3.1 Your Interest, Your Summaries

This paper proposes query focused video summarization based on user interest [14]. It improves relevance over generic summaries. It is aimed more at shorter videos than long multi speaker recordings.

2.3.2 Query Based Video Summarization with Pseudo Label Supervision

This work improves query based video summarization using pseudo labels under limited annotation settings [10]. It strengthens query to segment correspondence. The setting remains closer to generic short video summarization.

2.3.3 Video Summarization with Long Short Term Memory

This earlier work models video summarization as a sequence learning problem using LSTMs [13]. It established a useful temporal baseline. It predates modern multimodal language models and long context query answering.

2.4 Benchmarking and Evaluation

2.4.1 Comparing Large Language Models for Text Classification

This study compares open and closed language models across classification tasks and languages [3]. It shows that performance varies significantly across tasks. The study does not cover multimodal long video settings.

2.4.2 Breaking Down Video LLM Benchmarks

This work argues that many video benchmarks overestimate temporal understanding because models exploit shortcuts [1]. It separates knowledge, spatial, and temporal dimensions. This supports the need for stronger evaluation in time anchored systems.

2.4.3 Train Short, Infer Long

This work presents a Speech LLM for joint ASR and diarization on long audio [12]. It improves speaker continuity across segments. It remains audio only and does not extend to visual context.

2.5 Gaps and Motivation

2.5.1 Research Gaps

Current work remains fragmented across long video benchmarks, transcript based meeting summarization, query focused summarization, and model benchmarking. Very few systems combine speaker aware transcription, visual scene understanding, long range chronology, and query driven summarization in one open architecture [8, 6, 9].

2.5.2 Motivation

O.A.S.I.S is motivated by the need for an open source system that can process long recordings, align spoken content with visual context, and answer user queries in a time anchored manner. Existing work either focuses on transcripts, on benchmarks, or on individual components rather than a usable end to end pipeline. The proposed system aims to reduce manual video review, improve traceability of who said what and when, and create a practical platform for benchmarking open source models on realistic long form tasks [4, 5, 7]. This motivation directly follows from the combined gaps identified across current literature.

Chapter 3

Problem Statement and Objectives

This chapter states the problem addressed in the project and the specific objectives of the proposed prototype.

3.1 Problem Statement

Long form video content such as meetings, lectures, and documentaries is difficult to review efficiently. Existing tools often ignore visual context, fail to preserve long range chronology, or generate fixed summaries that do not answer user specific questions. This creates a need for an open source system combining audio and visual understanding with time anchored and query driven outputs.

3.2 Objectives

We set the following goals at the start of the project:

- **To benchmark the runtime performance and accuracy limitations of open source LLMs with chronological audio visual data.**

  This objective focuses on measuring latency, throughput, and model quality when open models process long multimodal inputs under the same conditions.

- **To develop a user query driven summarization engine that outputs structurally based on individual questions.**

  This ensures the system generates targeted answers instead of a single generic summary for every user and every video.

- **To integrate automated speaker indexing and time stamped transcription pipelines to map exactly who spoke and when.**

  This objective supports precise retrieval of speaker specific content and improves usability in meetings and lecture review.

- **To evaluate an architecture with visual scene transitions and speech analysis for a complete understanding.**

  This allows the system to connect spoken content with visual changes such as slides, screen shares, and scene transitions.

Chapter 4

Software Requirement and Specification

This chapter presents the functional requirements of the prototype along with the hardware and software needed to execute the system. The complete list of functional requirements is shown in Table 4.1. The required hardware and software specifications are listed in Table 4.2, and the VRAM usage at each pipeline stage is shown in Table 4.3.

4.1 Functional Requirements

Table 4.1: Functional Requirements for the O.A.S.I.S Prototype

ID	**Functional Requirement**
FR 01	The system must extract the audio track as a 16 kHz mono WAV file using FFmpeg.
FR 02	The system must split the input video into 30 second segments to preserve temporal continuity and simplify processing.
FR 03	The system must detect scene changes and extract representative keyframes from each chunk.
FR 04	The system must transcribe speech using WhisperX and produce timestamped transcript segments.
FR 05	The system must perform forced alignment to generate word level timestamps.
FR 06	The system must perform speaker diarization and identify distinct speakers across the recording.
FR 07	The system must assign each transcribed word to the correct speaker based on temporal overlap.
FR 08	The system must extract speaker names using regex, conversational heuristics, and LLM fallback when needed.
FR 09	The system must analyze extracted keyframes and generate short visual descriptions for each relevant frame.

ID	Functional Requirement
FR 10	The system must remove duplicate visual descriptions within the same chunk and preserve time range metadata.
FR 11	The system must combine transcript and visual context into a structured multimodal prompt.
FR 12	The system must use the same prompt structure across all benchmarked LLMs for fair comparison.
FR 13	The system must support audio only fallback when visual context is unavailable.
FR 14	The system must load benchmark models sequentially to avoid GPU memory overflow.
FR 15	The system must use quantized inference to execute larger open models within limited VRAM.
FR 16	The system must record benchmark metrics such as latency, throughput, peak VRAM, and output length.
FR 17	The system must display summaries, comparisons, and benchmark results through a Streamlit dashboard.

4.2 Hardware and Software Requirements

Hardware Specifications	Software Specifications
GPU: NVIDIA L4 with 24 GB VRAM	Operating System: Linux environment on Lightning AI Studio
System RAM: 16 GB minimum	Language: Python 3.12
Storage: 50 GB for models and video data	Deep Learning: PyTorch 2.x
CPU: 4 cores minimum	Quantization: bitsandbytes 0.45 or above
Platform: GPU accelerated cloud instance	Model Hub: Hugging Face Transformers 4.50 or above, ASR: WhisperX, Diarization: Pyannote Audio 3.x, Vision: Moondream2, Video Processing: FFmpeg 6.x, Dashboard: Streamlit 1.x, Version Control: Git

Table 4.2: Hardware and Software Specifications

Stage	Model	VRAM Usage
Transcription	WhisperX (small)	2 GB
Alignment	wav2vec2	1 GB
Diarization	Pyannote	1 GB
Name Extraction	Qwen 2.5 7B, 4 bit	5 GB
Visual Understanding	Moondream2 1.8B	4 GB
LLM Benchmarking	Each 7B to 8B model, 4 bit	5 to 6 GB

Table 4.3: VRAM Usage per Pipeline Stage

4.3 VRAM Usage Per Stage

Note: All models are loaded sequentially with explicit VRAM cleanup between stages, allowing the full pipeline to run on a single GPU.

Chapter 5

Proposed Methodology for Building Prototype

This chapter presents the methodology followed for building the O.A.S.I.S prototype. The complete system is organized as a six stage pipeline, where each stage produces structured output for the next stage. The overall architecture and data flow are illustrated in Figure 5.1. The benchmarked models are listed in Table 5.1, and the major design decisions are summarized in Table 5.2.

5.1 Overview

The prototype implements a fully open weight multimodal video understanding pipeline consisting of preprocessing, speech transcription with speaker diarization, visual scene understanding, multimodal fusion, comparative LLM benchmarking, and dashboard based visualization. The system is designed to run entirely on open source models without relying on proprietary APIs. The benchmarked models are listed in Table 5.1, and the key design decisions are summarized in Table 5.2.

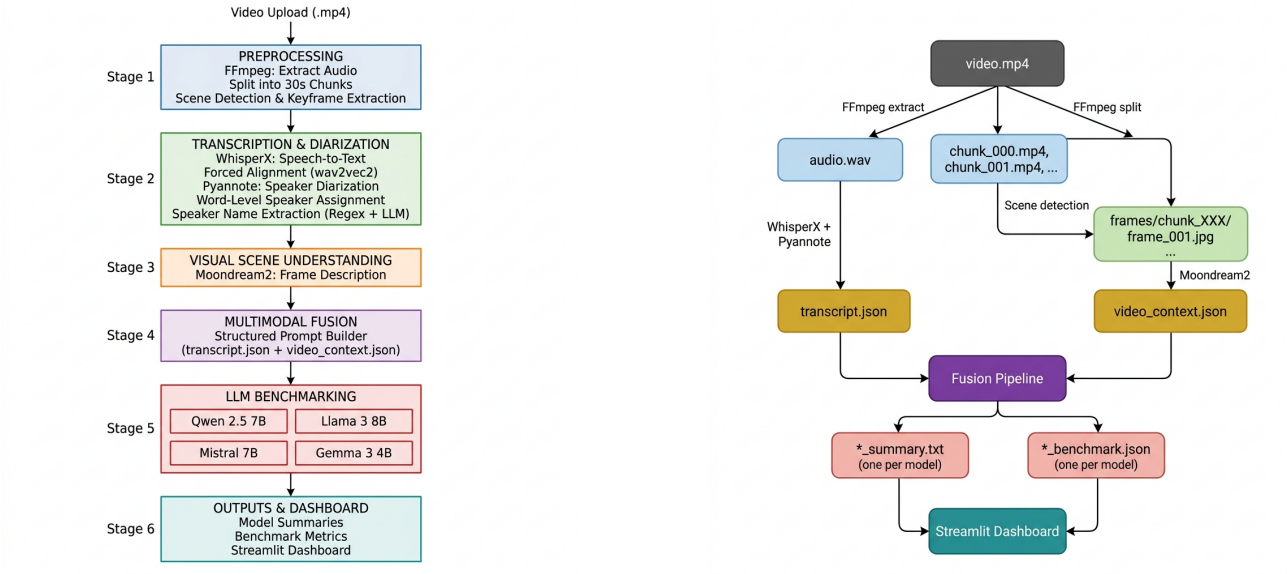
5.2 System Architecture

The architecture follows a sequential pipeline design. It begins with video preprocessing through FFmpeg, proceeds through parallel audio and visual analysis tracks, merges both outputs in a multimodal fusion stage, and ends with model benchmarking and dashboard visualization.

5.3 Step by Step Methodology

5.3.1 Stage 1: Video Preprocessing

The preprocessing stage extracts the audio track as a 16 kHz mono WAV file, splits the video into 30 second chunks, and identifies scene changes using FFmpeg based scene detection. Representative keyframes are then extracted from each chunk for downstream visual analysis.



(a) System Architecture Flowchart (b) Data Flow Diagram

Figure 5.1: System Architecture and Data Flow

5.3.2 Stage 2: Transcription and Speaker Diarization

This stage uses WhisperX for transcription, wav2vec2 for forced alignment, and Pyannote for speaker diarization. Word level speaker assignment is then performed so that each spoken word is mapped to the correct speaker at the correct timestamp. A three layer strategy based on regex, dialogue heuristics, and LLM fallback is used for speaker name extraction.

5.3.3 Stage 3: Visual Scene Understanding

In this stage, extracted keyframes are passed to Moondream2, which generates concise natural language descriptions of the scene. Duplicate descriptions within the same chunk are filtered and then grouped by time range.

5.3.4 Stage 4: Multimodal Fusion

The pipeline loads both transcript and visual context JSON files and builds a structured fusion prompt. This prompt includes speaker names, timestamped transcript segments, timestamped scene descriptions, and instructions for the model. If visual context is unavailable, the system falls back to audio only summarization.

5.3.5 Stage 5: Comparative LLM Benchmarking

Each language model is loaded one at a time using 4 bit quantization to stay within GPU memory limits. The same fusion prompt is used across all models for a fair comparison. After

inference, the model is deleted and GPU memory is explicitly cleared before loading the next one.

5.3.6 Stage 6: Dashboard Visualization

A Streamlit dashboard is used to visualize generated summaries, compare model outputs side by side, and inspect benchmark metrics. This stage makes the output interactive and easier to interpret for the user.

5.4 Models Benchmarked

Model	Parameters	Quantization	Context Window
Qwen 2.5 7B Instruct	7.6B	4 bit NF4	32K tokens
Llama 3 8B Instruct	8B	4 bit NF4	8K tokens
Mistral 7B Instruct v0.3	7.2B	4 bit NF4	32K tokens
Gemma 3 4B Instruct	4B	4 bit NF4	8K tokens

Table 5.1: Benchmarked LLM Models

5.5 Metrics Captured Per Model

The following metrics are recorded during benchmark runs:

- **latency_sec:** total inference time from model load to decoded output.
- **tokens_per_second:** generation throughput during output creation.
- **peak_vram_mb:** maximum GPU memory consumed during inference.
- **summary_length:** length of the generated answer or summary.
- **input_mode:** whether the run used multimodal input or audio only fallback.

5.6 Key Design Decisions

Decision	Rationale
WhisperX over Whisper	Word level alignment improves speaker assignment accuracy at boundary points.
Three layer speaker naming	Regex, heuristics, and LLM fallback provide a reliable naming pipeline.
Moondream2	It is small enough to fit within memory limits while remaining useful for scene description.
4 bit NF4 quantization	It reduces VRAM usage enough to benchmark larger open models on one GPU.
Sequential model loading	It avoids memory contention and ensures stable execution.
JSON intermediate files	They preserve timestamps, speakers, and chunk metadata clearly between stages.
Uniform prompt structure	It ensures a fair model comparison across all runs.

Table 5.2: Key Design Decisions

Chapter 6

GitHub Repository Details

The O.A.S.I.S project is maintained in a GitHub repository to support version control, reproducibility, collaboration, and future extension of the prototype. The repository contains the implementation of the multimodal pipeline and serves as the central location for the project source code.

Repository Link: https://github.com/Team007-FYP/O.A.S.I.S.git

Chapter 7

Annexures

This chapter contains supplementary material relevant to the prototype.

7.1 Definition of Prototype

A prototype is an early functional version of a system built to test a concept, validate design choices, and demonstrate core features before full scale deployment. In software projects, it is used to verify feasibility, expose limitations, and guide future refinement. In the context of O.A.S.I.S, the prototype validates that an open source multimodal pipeline can process long form video, align spoken content with visual context, and generate query driven outputs without depending on proprietary systems.

7.2 Definition of Functional Requirements

Functional requirements define the exact operations and services that a system must perform to satisfy user needs. For O.A.S.I.S, they describe the expected behavior of each stage of the pipeline.

- They must be specific and unambiguous.
- They must be testable through implementation output.
- They must map directly to system modules.

7.3 Sample Fusion Prompt Structure

The fusion prompt combines the participant list, timestamped transcript, timestamped visual context, and analysis instructions into a single structured input for the selected language model.

7.4 Example JSON Output Structure

transcript.json


```
{
  "metadata": {
    "num_speakers": 4,
    "total_segments": 81
  },
  "speakers": {
    "SPEAKER_00": "Tyler",
    "SPEAKER_01": "Tripp",
    "SPEAKER_02": "Beth",
    "SPEAKER_03": "Paul"
  },
  "segments": [
    {
      "id": 1,
      "start": 9.69,
      "end": 11.35,
      "speaker_id": "SPEAKER_03",
      "speaker": "Paul",
      "text": "Beth has joined the meeting."
    }
  ]
}
```


video_context.json


```
{
  "metadata": {
    "total_chunks": 7,
    "total_frames_analyzed": 78
  },
  "chunks": [
    {
      "chunk_id": "chunk_000",
      "time_range": "0:00 to 0:30",
      "descriptions": [
        {
          "frame": "frame_001.jpg",
          "description": "A person sitting at a desk in a video conference."
```

```
        }
      ]
    }
  ]
}
```

7.5 Repository Folder Structure

```
oasis/
|-- preprocess.py
|-- transcribe.py
|-- moondream_context.py
|-- fusion_pipeline.py
|-- model_runner.py
|-- dashboard.py
|-- requirements.txt
|-- README.md
|-- LICENSE
|-- audio/audio.wav
|-- chunks/chunk_000.mp4
|-- chunks/chunk_001.mp4
|-- frames/chunk_000/frame_001.jpg
|-- output/transcript.json
|-- output/video_context.json
‘-- output/benchmarks.json
```

Bibliography

[1] Breaking Down Authors. Breaking down video llm benchmarks: Knowledge, spatial, temporal. https://machinelearning.apple.com/research/breaking-down, 2024. Apple Machine Learning Research.

[2] Generating Minutes Authors. Generating minutes of meeting using llm. https://zenodo.org/records/13269298, 2023. Zenodo.

[3] J. Heseltine and co authors. Comparing large language models for text classification: Model selection across tasks, texts, and languages. https://sciety.org/articles/activity/10.31219/osf.io/jvgc5_v2, 2024. OSF.

[4] HourVideo Authors. Hourvideo: 1-hour video-language understanding. https://arxiv.org/abs/2411.04998, 2024. arXiv preprint arXiv:2411.04998.

[5] LongVideoBench Authors. Longvideobench: A benchmark for long-context interleaved video-language understanding. https://arxiv.org/abs/2407.15754, 2024. arXiv preprint arXiv:2407.15754.

[6] MeetingBank Authors. Meetingbank: A benchmark dataset for meeting summarization. https://arxiv.org/abs/2305.17529, 2023. ACL 2023.

[7] MISP-Meeting Authors. Misp-meeting: A real-world dataset with multimodal cues for long-form meeting transcription and summarization. https://aclanthology.org/2025.acl-long.753/, 2025. ACL 2025.

[8] OASIS Team. Oasis: On-demand hierarchical event memory for streaming video reasoning. https://arxiv.org/abs/2604.17052, 2026. arXiv preprint arXiv:2604.17052.

[9] QMSum Authors. Qmsum: A new benchmark for query-based multi-domain meeting summarization. https://arxiv.org/abs/2104.05938, 2021. arXiv preprint arXiv:2104.05938.

[10] Query-Based Authors. Query-based video summarization with pseudo label supervision. https://arxiv.org/abs/2307.01945, 2023. arXiv preprint arXiv:2307.01945.

[11] Scaling Up Authors. Scaling up video summarization pretraining with large language models. https://doi.org/10.48550/arXiv.2404.03398, 2024. arXiv preprint arXiv:2404.03398.

[12] Train Short Authors. Train short, infer long: Speech-llm enables zero-shot streamable joint asr and diarization on long audio. https://www.microsoft.com/en-us/research/publication/train-short-infer-long-speech-llm-enables-zero-shot-streamable-joint-asr-and-diariz 2024. Microsoft Research.

[13] Video Summarization Authors. Video summarization with long short-term memory. https://arxiv.org/abs/1605.08110, 2016. arXiv preprint arXiv:1605.08110.

[14] Your Interest Authors. Your interest, your summaries: Query-focused long video summarization. https://arxiv.org/abs/2410.14087, 2024. arXiv preprint arXiv:2410.14087.